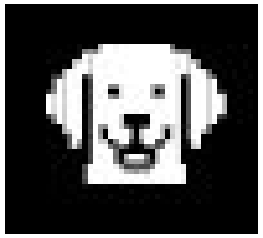
Workshop - Spec Coding with OpenRAG

David Jones-Gilardi
Developer Relations Engineer
david.gilardi@ibm.com

IBM



-> -> ->

If you haven't already downloaded and installed OpenRAG...

...Please do so now

There is no shame in this 😡 😱

https://ibm.biz/~9xgRz7OtH

Agenda

Timeline

Introduction	RAG and Spec Coding!
Use Case	What problem are we solving?
Install	Install OpenRAG (if you haven't done it already)
Data Loading	Ingesting data into OpenRAG
Workflow	Use your AI coding agent tool to build services that access the data.
Closing	Finishing remarks and resources.

David Jones-Gilardi

> Developer Relations Engineer @ IBM

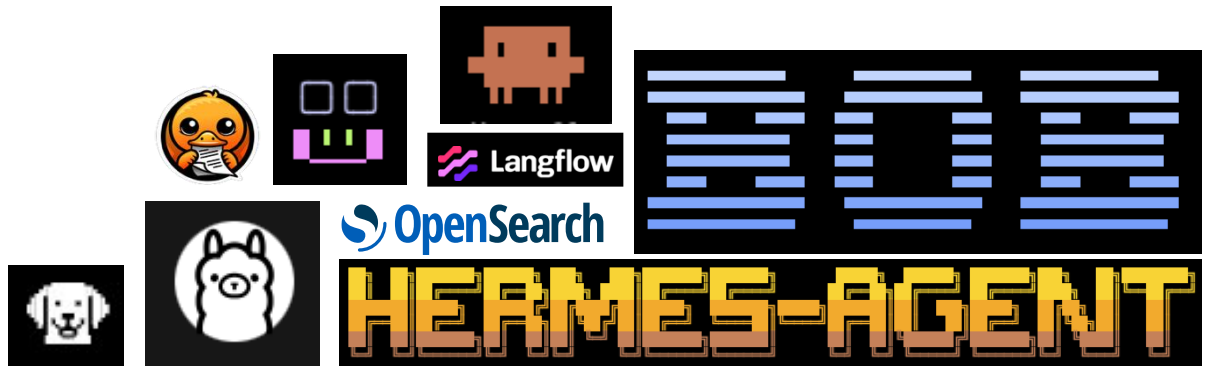
```
$ cat profile.yaml
experience: "30+ years"
sweet_spots:
  - AI agents & RAG
  - context engineering
  - many coding languages
- spec coding (do I code any more?)
# also ships outside work:
  - ninja warrior
  - pixel art roguelite
  - home improvement
```

$ linkedin -> in/david-gilardi

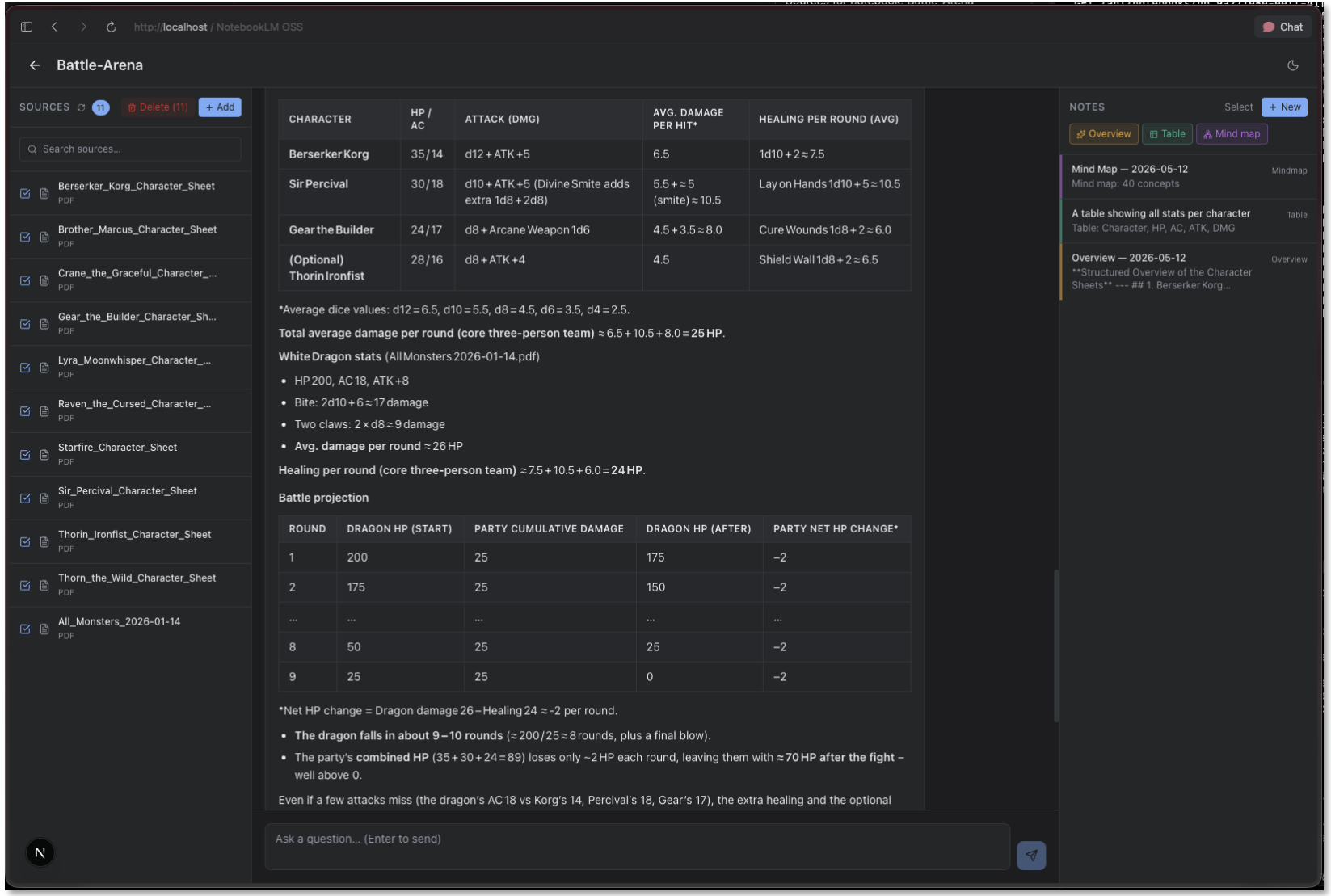
$ github -> SonicDMG

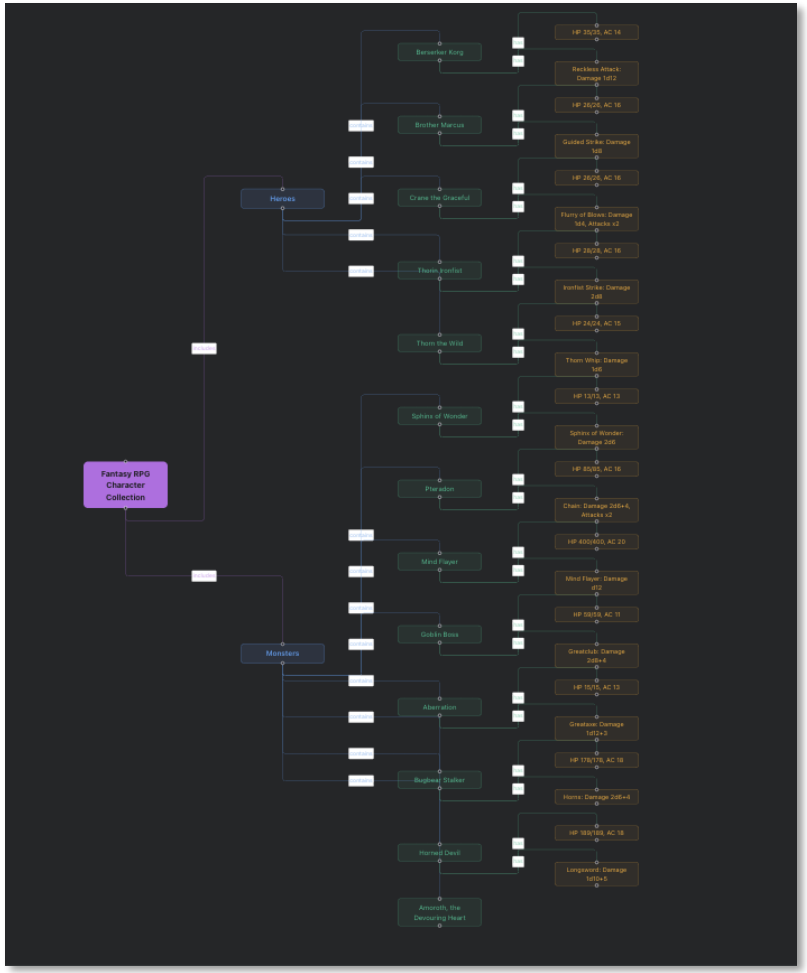
$ x -> @SonicDMG

$ site -> sonicdmg.github.io



notebooklm_oss





Spec first, code second.

Write detailed specifications — requirements, data models, API contracts — and use them as executable blueprints for an AI coding agent. The spec is your source of truth. The agent generates the code.

THE KEY INSIGHT

Planning matters even more with an agent. Iterate on the plan first, then hand it off. The spec is the first artifact you and the AI build together.

NOT VIBE CODING

No “just make it work” prompting. You define what right looks like **before** any code is written.

NOT WATERFALL

Each step is iterative. Refine requirements, sharpen the contract, adjust tasks — as many loops as you need.

THIS WORKSHOP

Three steps: requirements, spec (design + OpenAPI), then tasks + code. You’ll build a real app using this workflow.

“Spec-driven development is one of the most important practices to emerge in 2025.” — Thoughtworks Technology Radar

“RAG is so 2024, why are we talking about this?”

RAG is dead....

long live RAG!!!!

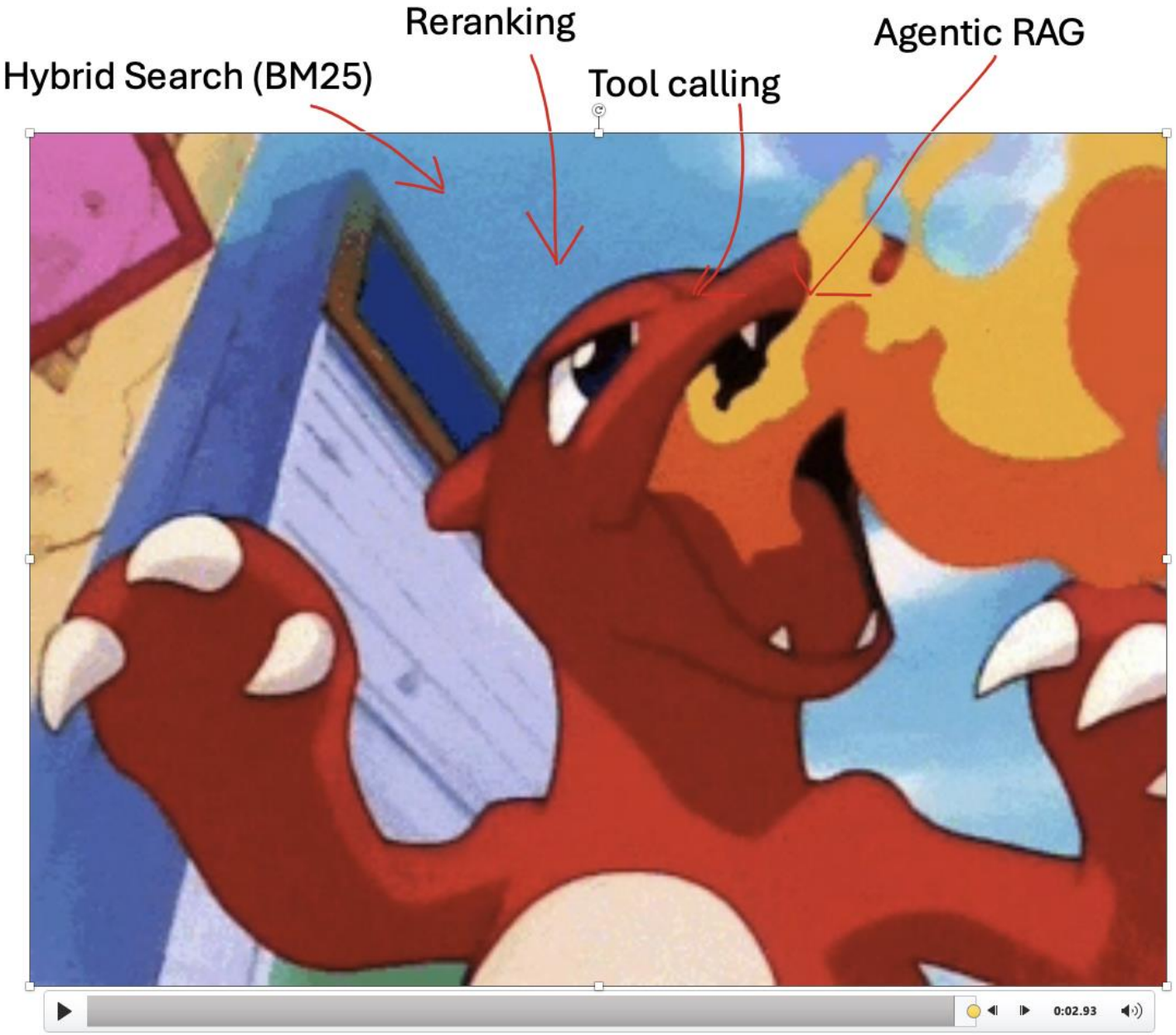
OpenRAG

RAG has Evolved

2024 →

RAG has Evolved

Now →



This is not a story about

RAG > Direct Context

This IS a story about...


Hybrid
Direct
Context
RAG
osamot.tumblr.com

Direct Context

Context Rot

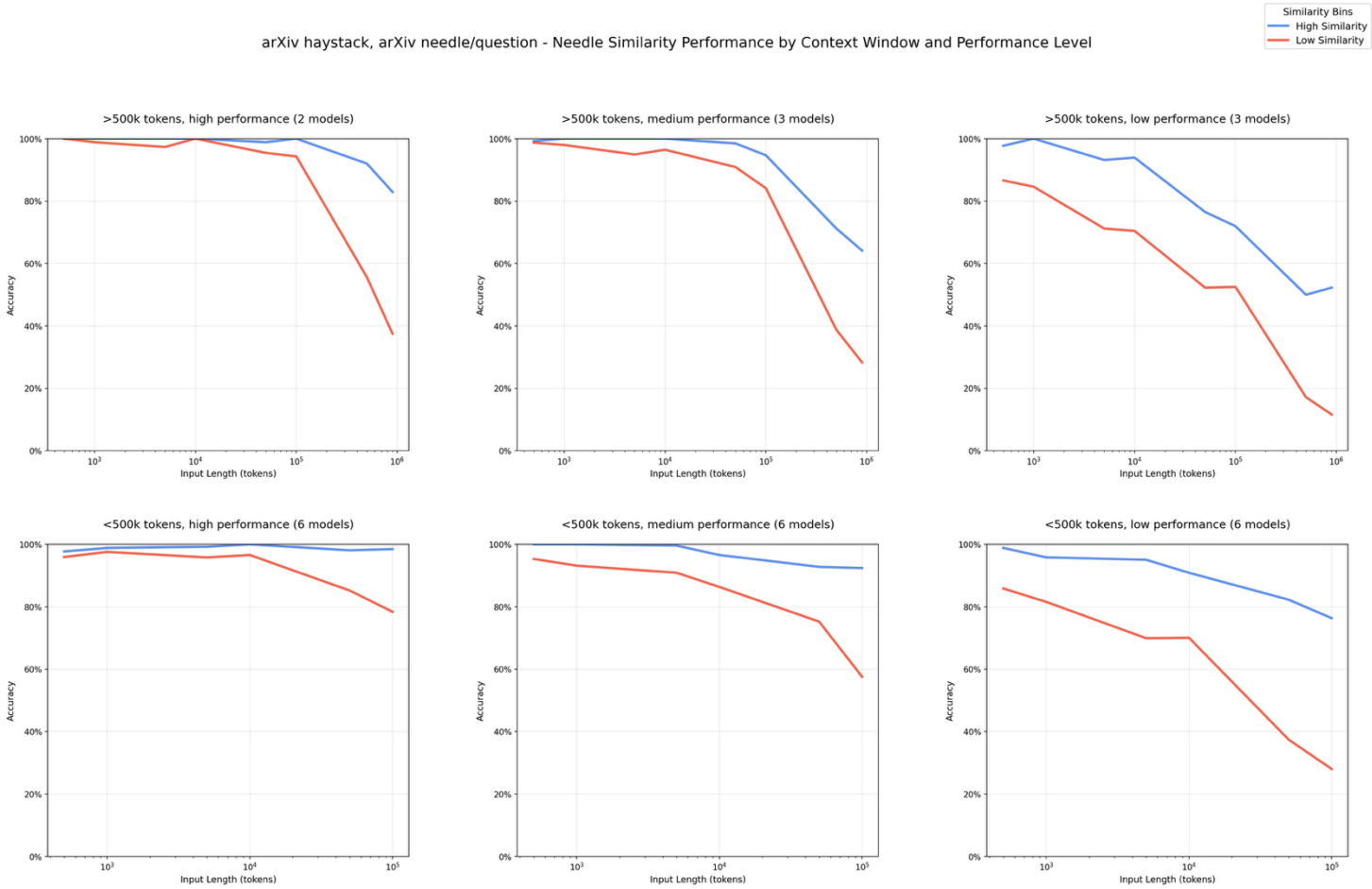
A model might technically handle 128K tokens

BUT

Performance and accuracy fall off drastically outside of effective context range

For most models, effective context = ~30k – 50k tokens

Some can push to ~300k, but reasoning ability starts to drop (Claude, Gemini, Codex)



RAG vs Direct Context

RAG

Great for storing large sets of data

Supports large documents

Supports many file formats

Low overall token cost

Pay for “cost” upfront, then costs diminish

Direct Context

Maximum accuracy*

Fast inference*

Smaller documents

Hard limit on num tokens

Uses more tokens

Pay token cost on every ingest/query

Not shared across users

*Within “effective” context

Agentic RAG

Traditional RAG breaks down

Static retrieval gives inconsistent results

Easy to over - or under - fetch context

One-shot answers can't handle reasoning

No validation or self-correction

Agentic RAG does more

Decides what it actually needs

Retrieves in steps, not one shot

Validates before returning an answer

Reasons over your data, not just fetches

You're building a context layer, not a retrieval layer.
"Agentic RAG is the researcher who thinks, rather than the intern who just follows orders." — Aram Radif

OpenRAG

IBM's open-source RAG distribution
https://www.openr.ag/

Ingest: Docling

Parses any format (PDF, DOCX, HTML)
Handles messy, real-world data
Structures content for retrieval

Retrieve: OpenSearch

Vector + semantic search
1M+ downloads, production-grade
Advanced indexing at any scale

Orchestrate: Langflow

Drag-and-drop workflow builder
Re-ranking and filtering
Multi-agent coordination

```
$ uvx openrag
```

4k+ GitHub stars | From documents to agentic search in minutes

Docling

The Ingest Layer
https://github.com/DS4SD/docling

What it does

- Intelligent document parsing for any format
- Handles PDFs, DOCX, HTML, images, tables
- Preserves structure: headings, lists, code blocks
- Outputs clean, structured content ready for RAG
- IBM open-source project, production-proven

Why it matters

- Real-world documents are messy
- Copy-paste RAG pipelines break on tables and images
- Docling turns chaos into structured knowledge
- No manual preprocessing or format wrangling

OpenSearch

The Retrieval Engine
https://github.com/opensearch-project

What it does

- Vector search with semantic understanding
- Hybrid search: keyword + vector combined
- Advanced indexing for fast retrieval
- Scales from laptop to enterprise cluster
- Built-in dashboards and monitoring

Why it matters

- 1M+ downloads, battle-tested at scale
- Open-source, no vendor lock-in
- The foundation your RAG data lives on
- Production-grade out of the box

The Orchestration Layer
https://github.com/langflow-ai/langflow

What it does

- Visual drag-and-drop workflow builder
- Chain retrieval, re-ranking, and filtering
- Multi-agent orchestration out of the box
- Custom logic nodes for any pipeline
- Iterate visually, deploy via API

Why it matters

- No hand-wiring RAG pipelines in code
- See your entire retrieval flow at a glance
- Rapid experimentation without redeploying
- The brain connecting **Docling** and **OpenSearch**

A working app, a reusable workflow, and a repo full of proof.

A working app

Service layer backed by OpenRAG SDK, tested and demoed.

A repeatable workflow

Requirements, design, OpenAPI contract, tasks. Spec first, code second.

Evidence of progress

Tests pass, demo script runs, CI is green. Not just code — proof it works.

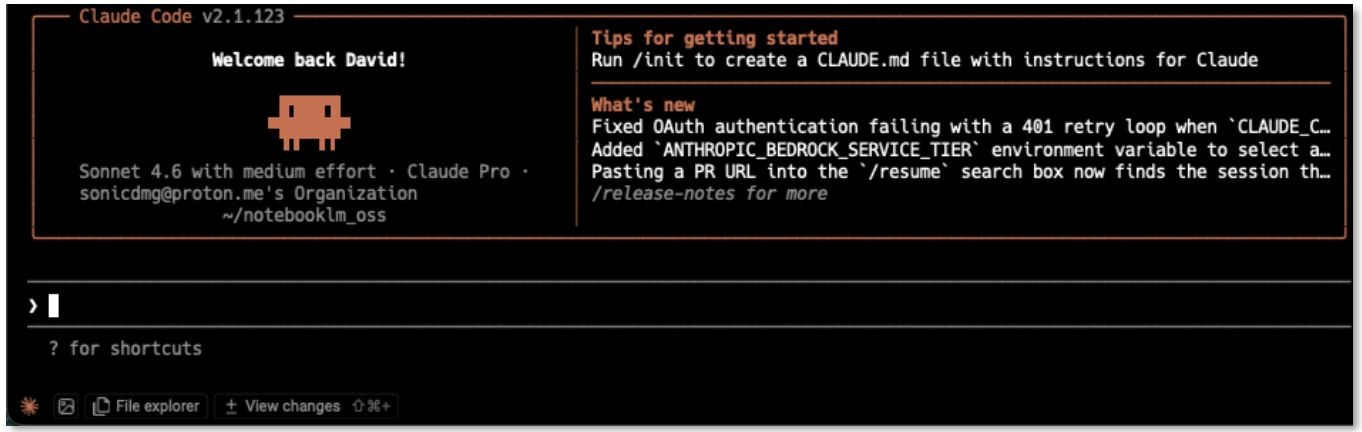
Agentic Notebook App

- Create personal "notebooks".
- "Talk" with data, create overviews, data tables, mind maps, etc...
- Allows CRUD operations notebooks and data.
- Serves restful endpoints to each underlying operation.

```
git clone https://github.com/SonicDMG/spec_coding_openrag_notebook_app
```

Agentic Notebook LM App Pre-requisites

- Python 3.13
- OpenRAG installed
- An agentic coding tool (Cursor, Claude Code, IBM Bob, GitHub CoPilot, etc...)
- Access to an LLM provider (OpenAI, Anthropic, Ollama, watson.x)



Exercise #1
Install OpenRAG

Install OpenRAG

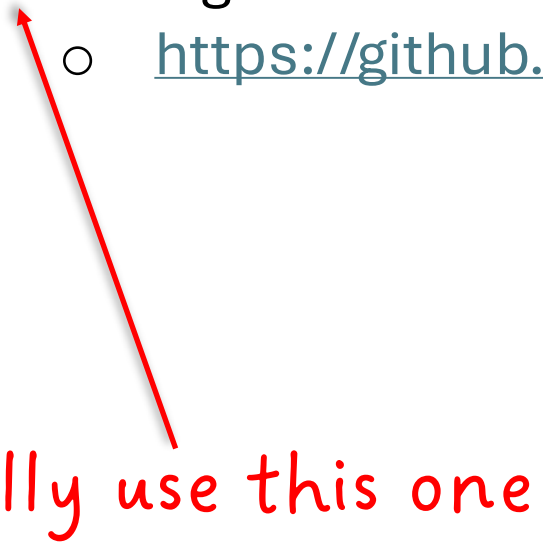
You have two options

1. Follow the install docs
    - https://docs.openr.ag/install-options
    - uvx openrag is generally the quickest
2. Use an agent SKILL
    - https://github.com/langflow-ai/openrag/tree/main/plugins

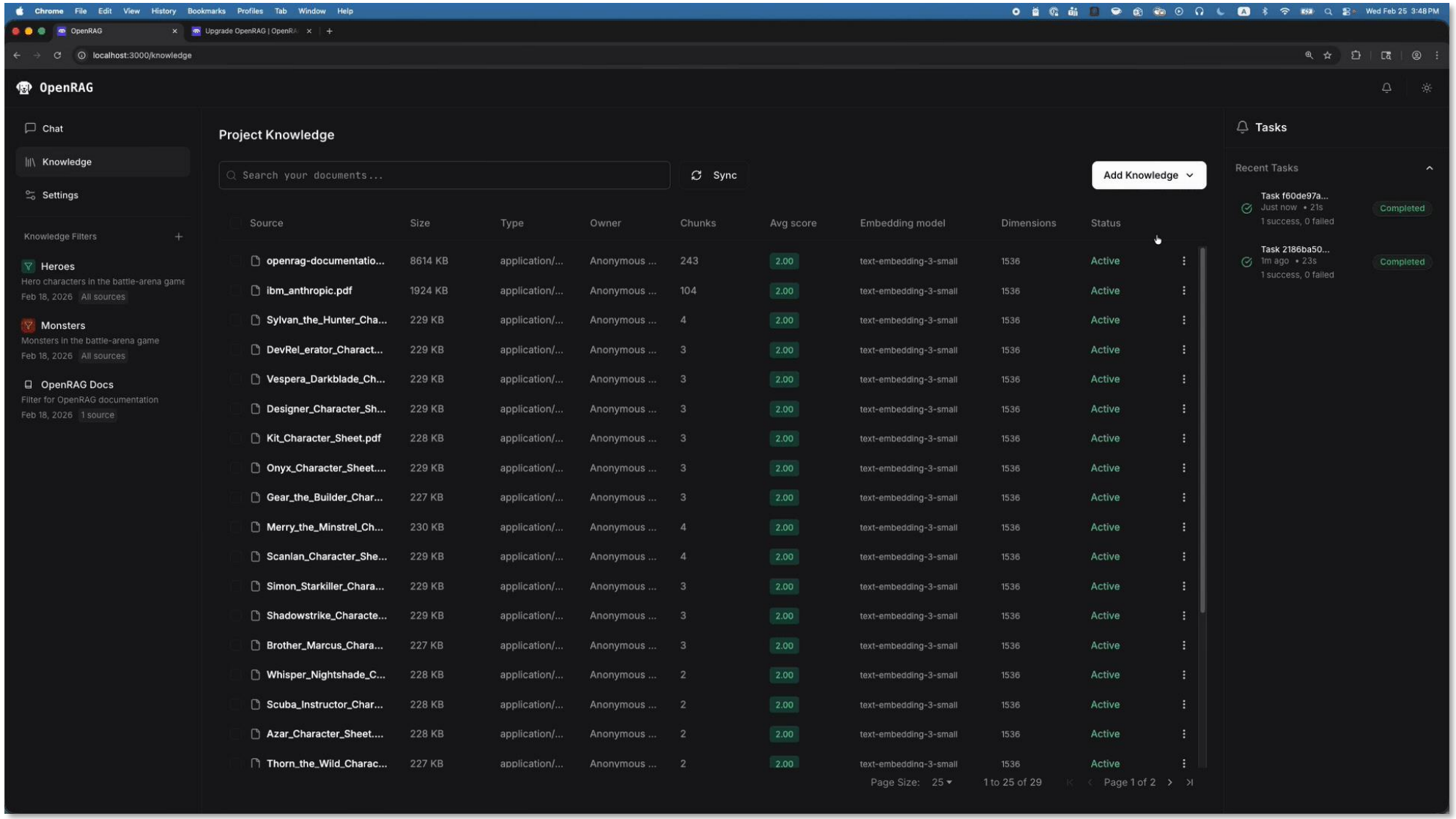
Install OpenRAG

You have two options

1. Follow the install docs
    - https://docs.openr.ag/install-options
    - uvx openrag is generally the quickest
2. Use an agent SKILL
    - https://github.com/langflow-ai/openrag/tree/main/plugins



Once complete, you should see something like...



Building the Service Layer

Three steps. Iterate each one until it’s right.

1 Requirements

Output `requirements.md`

↻ until specific, testable, complete

2 Spec

Output `design.md` `openapi.yaml`

↻ until spec validates + covers reqs

3 Tasks + Implement

Output `todo.md` `src/` `tests/`

↻ until all tests pass

What “iterate” looks like

Prompt the agent. Review the output. Refine the prompt. Run again.

Don’t move to the next step until you’re happy with the current output.

The spec files are your ground truth — they survive every iteration.

Goal + Context + Constraints + Examples = Awesome Prompt

Goal

What you want built/changed (definition of done). **Bonus:** Defined Role

Context

Stack + relevant code/files

Constraints

Must/never rules (deps, API, perf, style)

Examples

Inputs → outputs + edge cases

Pro Tip: Have the agent interview you. “You are a product manager. Interview me about X”

Context is everything

Better context = fewer wrong assumptions, less refactoring, faster correct output

Include only what matters:

- The relevant files / interfaces
- Docs
- Websites

MCP

Avoid if possible

Contributes to context bloat

Context7 MCP maybe an exception

Skills

Small bundles of docs + instructions

Can dynamically load MCPs

Easy to make

Pro Tip: Have the agent create a doc directory and scrape docs to create local docs on everything

Exercise #5a
Building the requirements.md file

Step 1 **Requirements** ↻ iterate until done

WHAT WE'RE DOING

Write a clear set of requirements for your project. Think user stories, acceptance criteria, and constraints. Give each requirement a testable ID (e.g. REQ-001). Avoid anything technical!

HOW TO PROMPT

```
# Tell the agent what you're building

I'm building a [your project]. Write requirements in requirements.md with:

- Numbered IDs (REQ-001, REQ-002 ...)
- Acceptance criteria for each
- Personas and User flows
- Do not include any implementation details such as code or technology choices
- This should be an MVP/demo level project, no production/enterprise level code, no security concerns, build only the most basic application
```

DONE =

`requirements.md` exists with numbered, testable requirements that you've reviewed and are happy with.

Pro Tip: Check for scope creep. That happens. A LOT

Finished early? Ask the agent to add edge cases or "what could go wrong" scenarios.

Exercise #5b
Building the design.md and OpenAPI.yaml files

Step 2 **Spec (Design + OpenAPI)** ↻ iterate until done

WHAT WE'RE DOING

Turn your requirements into a data model (design.md) and an OpenAPI contract (openapi.yaml). The OpenAPI file becomes the single source of truth for your API.

HOW TO PROMPT

```
# Feed it the requirements as context

Read requirements.md.  From those requirements:

1. Create design.md with data model with knowledge filters needed for OpenRAG.

2. Create openapi.yaml (OpenAPI 3.1) with all endpoints, schemas, and error responses

3. Map each endpoint back to a REQ-ID

4. Detail the technical requirements for frontend and backend

5. Ensure to use the openrag_sdk (0.3.1) skill for details on using the OpenRAG SDK

6. Ask questions regarding application tech stack to use for the specific type of app.
```

DONE =

`design.md` + `openapi.yaml` that validates and maps every requirement to an endpoint.

Finished early? Ask the agent to validate the OpenAPI spec and add example responses for each endpoint.

What you should have now

Your repo

- ✅ `requirements.md`
- ✅ **`design.md`** (new)
- ✅ **`openapi.yaml`** (new)
- ✅ `.env`
- ✅ `.gitignore`

SAMPLE REFINEMENT PROMPT

```
Validate openapi.yaml.  Check that
every REQ-ID in requirements.md is
covered by at least one endpoint.
List any gaps and fix them.
```

Next up → Step 3: Tasks + Implement (todo.md + working code)

Exercise #5c
Build + Execute the todo.md file

Step 3 **Tasks + Implement** ↻ iterate until done

WHAT WE'RE DOING

Break the spec into a task list (todo.md), then have the agent implement each task: server, frontend, Astra DB integration, and tests. Run tests after each task.

HOW TO PROMPT

```
# Give it all the context it needs

Read requirements.md, design.md, openapi.yaml.

1. Create todo.md breaking the spec into tasks

2. Implement each task.  Use .env for creds.

3. Write tests and run them after each task

4. Mark each task done in todo.md when it passes
```

DONE =

A working server + frontend connected to OpenRAG. All tests passing. Every task checked off in `todo.md`.

Pro Tip: Use github issues instead of a file. Using the gh command line tool is extra bonus tip

Finished early? Ask the agent to add error handling, improve the UI, or write integration tests against OpenRAG.

What you should have now

Your repo

- ✅ `requirements.md`
- ✅ `design.md`
- ✅ `openapi.yaml`
- ✅ **`todo.md`** (new)
- ✅ **`src/`** **`tests/`** (new)

SAMPLE REFINEMENT PROMPT

```
Run all tests.  For any that fail:
1. Read the error output
2. Check openapi.yaml for the expected
   behavior
3. Fix the implementation, not the test
4. Re-run until green
```

You did it! A working app, a reusable workflow, and a repo full of proof — built with spec-driven development.

Thank you!